

Visual Aerial Navigation (VAN) System

Stereo Visual-SLAM Pipeline Implementation & Analysis

Nir Ellor Waizner, Ayala Houri

Timeline and contact Email: 18.08, 20.08. 23.08, ayala.houri@mail.huji.ac.il

https://github.com/NirEllor/VAN_ex

1. Introduction and Overview

Background

Visual Simultaneous Localization and Mapping (Visual SLAM) estimates a platform's 3D motion trajectory using visual data from an onboard camera system. In this project, we address ego-motion estimation from a sequence of images captured at 10 frames per second by an onboard stereo camera. The system tracks 2D features across frames, triangulates 3D spatial points, calculates relative vehicle poses via PnP-RANSAC, and bounds cumulative drift using local Bundle Adjustment (BA), Pose-Graph factor optimization, and Loop Closure detection.

Importance of the Problem

- **Infrastructure Independence (GPS-Denied Navigation):**
Standard navigation systems rely heavily on satellite positioning (GPS/GNSS). However, GPS signals frequently degrade or fail entirely in environments such as urban canyons, tunnels, covered parking facilities, or dense forests. Visual SLAM provides a self-contained, infrastructure-free localization solution that operates purely on local visual observations.
- **Cost-Effective & High-Information Sensing:**
Compared to expensive, heavy active sensors like LiDAR, stereo camera rigs are lightweight, passive, low-power, and cost-effective. They capture rich spatial and visual context that can be used simultaneously for localization, obstacle detection, and environmental mapping.
- **Safety and Autonomy in Mobile Robotics:**
Accurate real-time localization and trajectory consistency are fundamental requirements for path planning, collision avoidance, and safe execution in autonomous driving and mobile robotics.

2. Code Organization

Project Structure:

The codebase is structured modularly, separating concerns into geometry calculations, database tracking, bundle adjustment, factor-graph optimization, and visualization routines. Pipeline execution is driven by `main.py`, while ablation studies and experiment scripts reside in `experiments`

```

code/
├── tracking_database.py      (class Observation @22, class TrackingDB @30)
├── homework/                (legacy exercises; van_utils.py fallback)
├── project/
│   ├── main.py              (pipeline driver)
│   ├── outputs/             (generated report figures)
│   ├── experiment_results/   (ablation study result pickles)
│   ├── utils/
│   │   └── geometry.py      (triangulation, PnP-RANSAC, KITTI/Rodrigues
formulas)
│   │   ├── tracking.py      (build_data pipeline, inline RANSAC)
│   │   ├── bundle_adjustment.py (solve_bundle_window - GTSAM solving)
│   │   └── pose_graph.py    (build/optimize pose graph, relative
pose+covariance)
│   │   └── loop_closure.py  (candidate detection, verification, factor
integration)
│   │   └── visualization.py (all 31 plot_* functions)
└── experiments/             (5 compare_*.py ablation scripts + harness)

```

System Overview

- **1. Triangulation:** Stereo point triangulation using rectified image pairs and camera calibration | Function: `triangulate_points_opencv` | File: `code/project/utils/geometry.py` | Line: 208
- **2. RANSAC:** PnP outlier rejection via RANSAC consensus | Function: `run_custom_pnp_ransac` | File: `code/project/utils/geometry.py` | Line: 237
- **3. PnP trajectory calculation:** Temporal pose estimation from track observations | Function: `build_data` | File: `code/project/utils/tracking.py` | Line: 127
- **4a. Database definition (Observation):** Observation data structure for track observations | Function: `class Observation` | File: `code/tracking_database.py` | Line: 22
- **4b. Database definition (TrackingDB):** TrackingDB data structure for tracking | Function: `class TrackingDB` | File: `code/tracking_database.py` | Line: 30
- **5. Adding a frame to the database:** Feature track integration across frames | Function: `update_tracks` | File: `code/tracking_database.py` | Line: 83
- **6. Single Bundle creation:** Single-window pose and landmark optimization via GTSAM | Function: `solve_bundle_window` | File: `code/project/utils/bundle_adjustment.py` | Line: 235
- **7. Relative transformation + covariance:** Pose constraint extraction with uncertainty quantification | Function: `compute_relative_pose_and_covariance` | File: `code/project/utils/pose_graph.py` | Line: 150
- **8a. PoseGraph building (init):** Factor graph construction and initialization | Function: `build_and_initialize_pose_graph` | File: `code/project/utils/pose_graph.py` | Line: 77
- **8b. PoseGraph building (optimize):** Factor graph optimization | Function: `optimize_pose_graph` | File: `code/project/utils/pose_graph.py` | Line: 105
- **9a. Loop closure detection:** Candidate detection via Mahalanobis distance | Function: `detect_loop_closure_candidates` | File: `code/project/utils/loop_closure.py` | Line: 42
- **9b. Loop closure factor creation:** Relative pose estimation and graph integration | Function: `add_loop_closures_and_optimize` | File: `code/project/utils/loop_closure.py` | Line: 229

3. Performance Analysis

The following section evaluates the proposed system from two complementary perspectives. First, controlled ablation studies investigate the influence of key algorithmic design choices on localization accuracy and computational efficiency. Second, the complete Visual-SLAM pipeline is evaluated using the quantitative metrics required by the project specification.

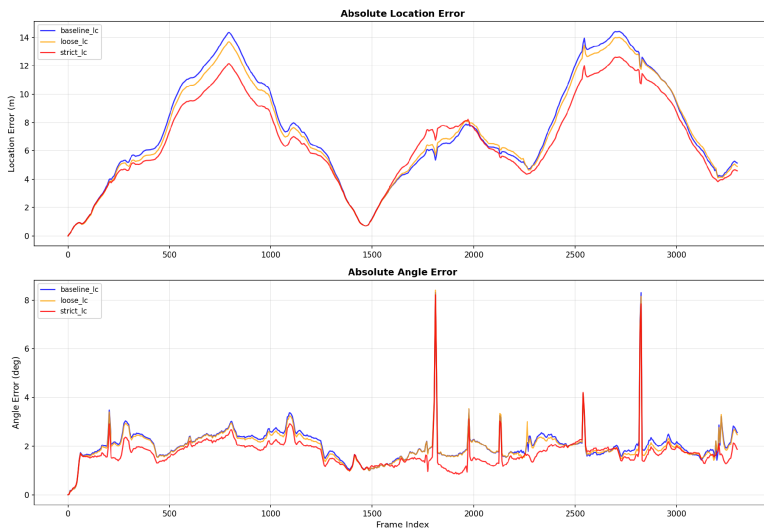
Experiments & Improvements

Five ablation studies systematically evaluate the impact of key system components. Loop closure is enabled **ONLY** for Experiment 1 (which specifically tests its gating parameters) and disabled for Experiments 2–5. This ensures a fair comparison, prevents result confounding from a variable number of detected loops, and isolates the pure effect of each tested parameter.

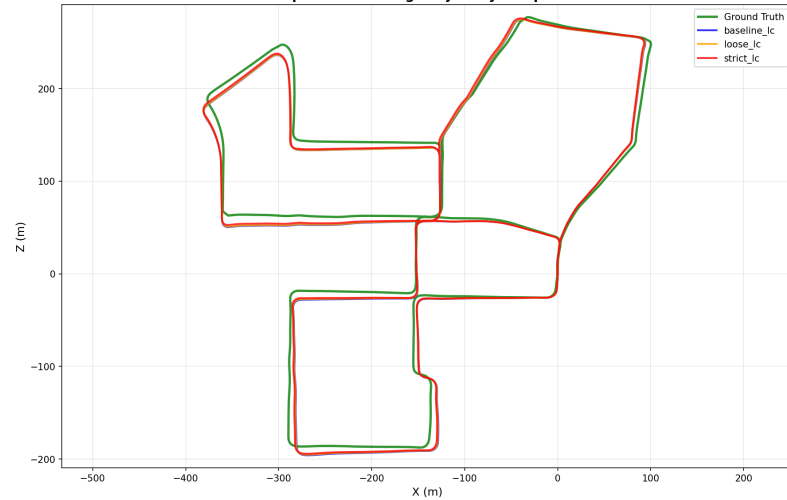
1. Loop-Closure Gating Strictness

Tests Mahalanobis distance threshold and inlier ratio threshold for candidate acceptance. Variants: strict (threshold=300, inlier_ratio=0.8), baseline (1000, 0.6), loose (3000, 0.4). LOOP CLOSURE: ENABLED (this experiment specifically tests loop-closure detection and gating parameters).

Loop-Closure Gating: Absolute Error



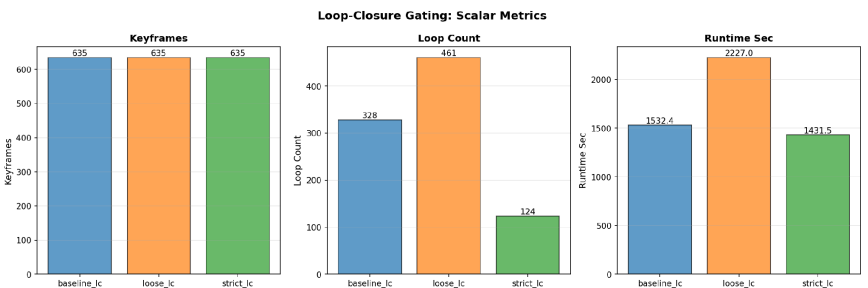
Loop-Closure Gating: Trajectory Comparison



Applying a strict gating threshold (strict_lc)

significantly improves overall system performance. Although strict gating accepts fewer loop closures (124 compared to 461 in loose_lc), it effectively eliminates false-positive loop acceptances that degrade trajectory accuracy and introduce optimization overhead.

Consequently, strict_lc achieves the lowest absolute location error while reducing runtime to a minimum (1431.5 seconds), demonstrating the optimal trade-off between localization precision and computational efficiency.



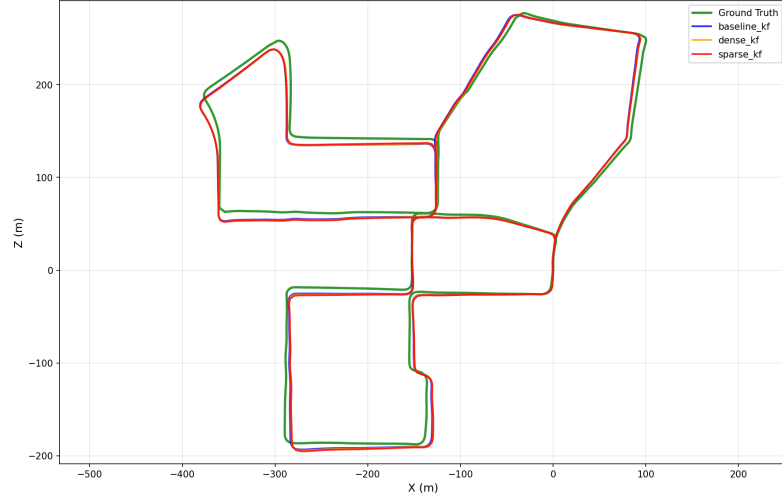
2. Keyframe Density

This experiment evaluated the impact of the keyframe selection distance threshold on trajectory accuracy and computational efficiency across three settings: dense (1.25 m), baseline (2.5 m), and sparse (5.0 m).

Keyframe Density: Absolute Error



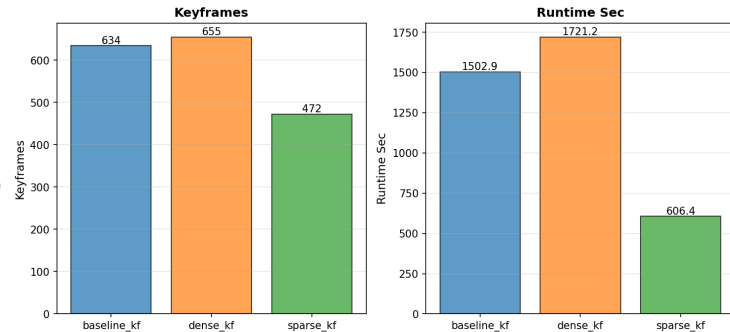
Keyframe Density: Trajectory Comparison



Keyframe density demonstrates a clear trade-off between localization accuracy and computational overhead. The sparse variant (sparse_kf) significantly reduced total runtime to **606.4**

seconds—representing a $\sim 60\%$ speedup compared to the baseline—while maintaining trajectory error comparable to the dense setup. However, baseline_kf (2.5 m) achieved the lowest overall Absolute Location Error. Consequently, sparse_kf provides an optimal configuration for resource-constrained, real-time applications, whereas baseline_kf remains preferable when trajectory precision is paramount.

Keyframe Density: Scalar Metrics

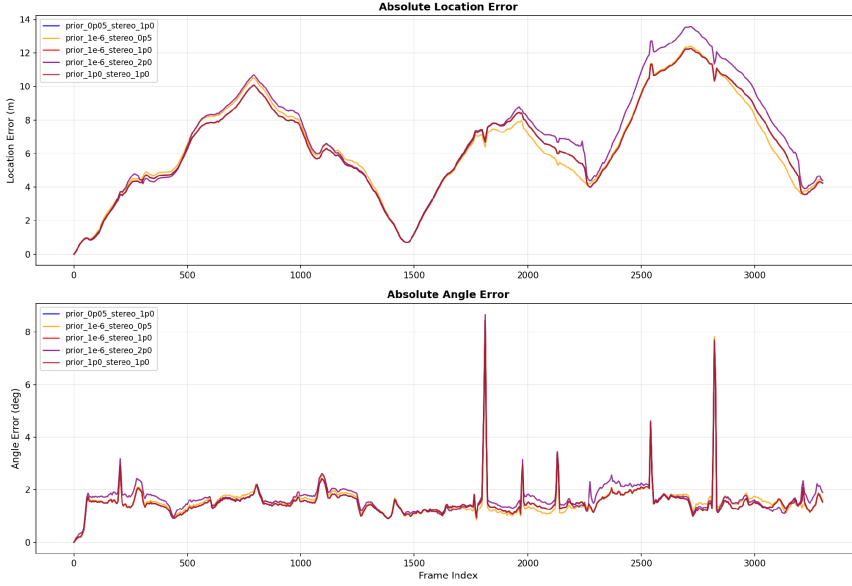


3. Bundle Adjustment Noise Model

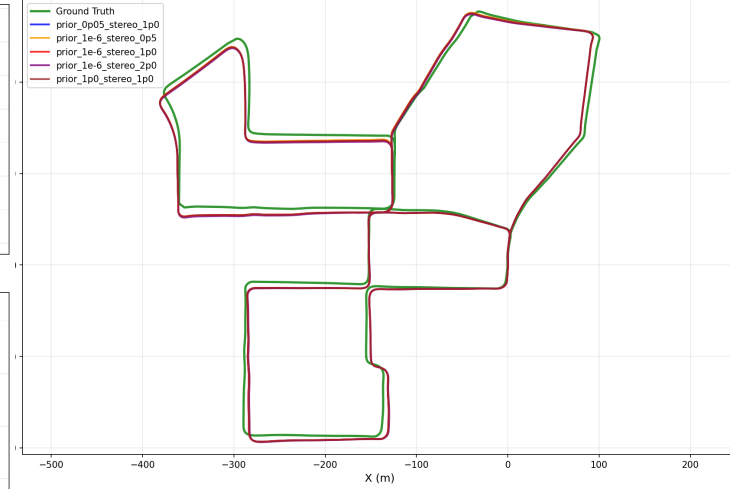
This experiment evaluated the impact of factor noise modeling (prior and stereo measurement sigmas in GTSAM) on trajectory estimation and optimization convergence.

Tests GTSAM factor weights via prior_sigma $\in \{1.0, 0.05, 1e-6\}$ and stereo_sigma $\in \{0.5, 1.0, 2.0\}$.

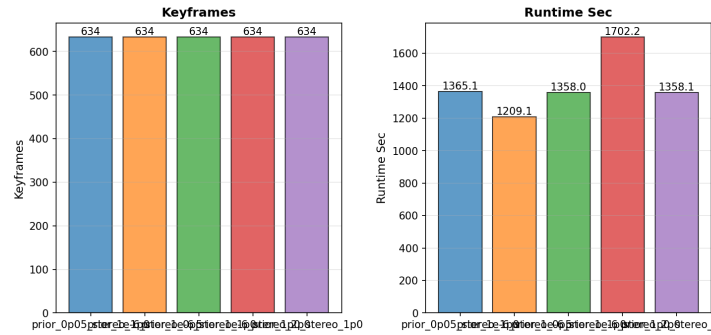
Bundle Noise Model: Absolute Error



Bundle Noise Model: Trajectory Comparison



Bundle Noise Model: Scalar Metrics



We can observe that all tested configurations produce virtually identical trajectory plots against the ground truth. However, quantitative evaluation reveals that assigning excessive uncertainty to stereo visual factors (stereo_sigma = 2.0) degrades system performance, leading to higher location error and an increased runtime of 1702.2 seconds. Conversely, tighter visual factor sigmas (stereo_sigma = 0.5 or 1.0) maintain stable and accurate trajectory estimates. The prior_1e-6_stereo_0p5 configuration yielded the optimal balance, achieving high localization precision while minimizing computational runtime to 1209.1 seconds.

4. PnP-RANSAC Parameters

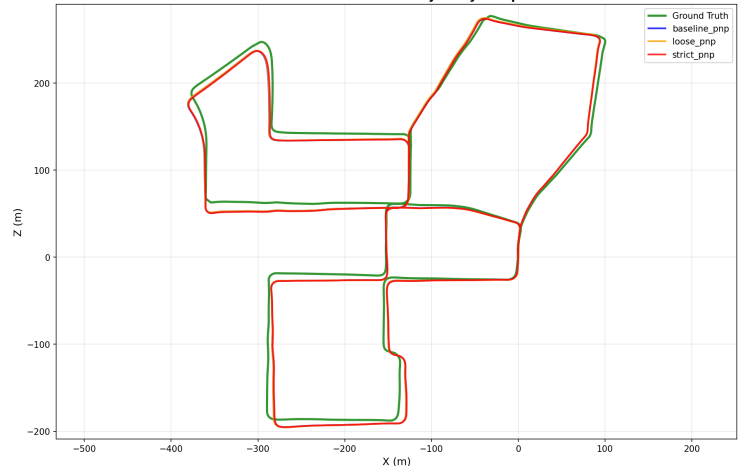
This experiment evaluated the impact of PnP-RANSAC settings, specifically the pixel inlier threshold and maximum iteration count, on pose estimation quality and overall runtime.

Tests inlier threshold (pixels) and iteration count. Variants: strict (1px, 100 iter), baseline (2px, 50), loose (4px, 25).

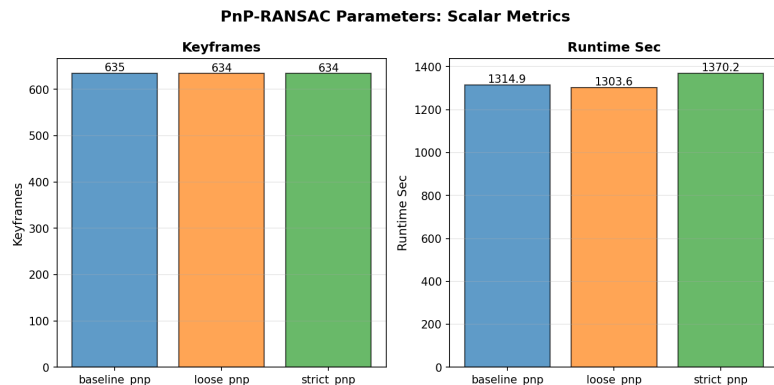
PnP-RANSAC Parameters: Absolute Error



PnP-RANSAC Parameters: Trajectory Comparison

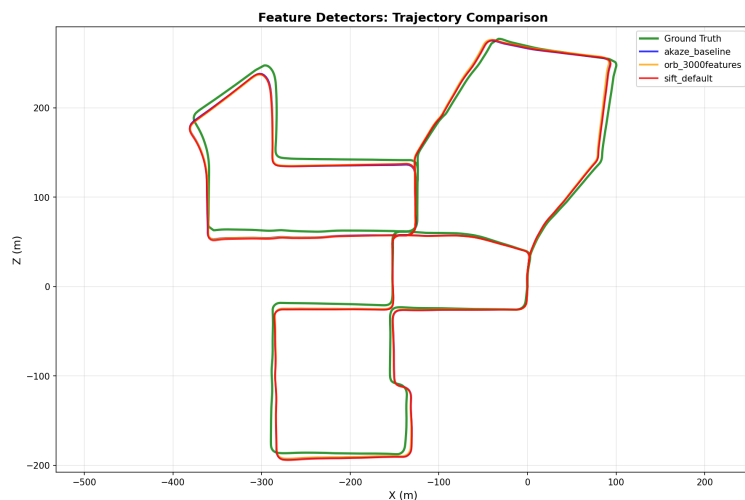
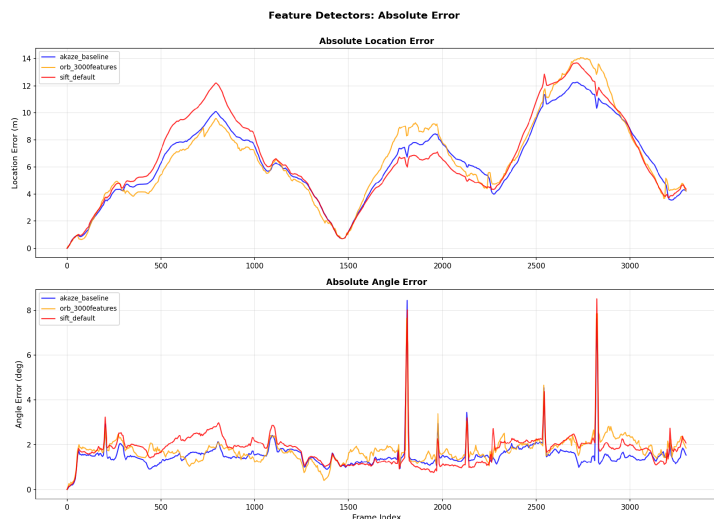


Visually, all tested configurations generate nearly identical trajectory estimations. Quantitatively, overly strict gating (strict_pnp: 1px threshold, 100 iterations) does not yield precision gains and slightly increases angular error due to excessive inlier rejection, while adding slight computational overhead (**1370.2 seconds**). Conversely, baseline_pnp (2px, 50 iterations) and loose_pnp (4px, 25 iterations) demonstrate robust tracking with lower runtimes, with baseline_pnp offering the most balanced trade-off between outlier rejection and inlier retention for reliable relative pose estimation.

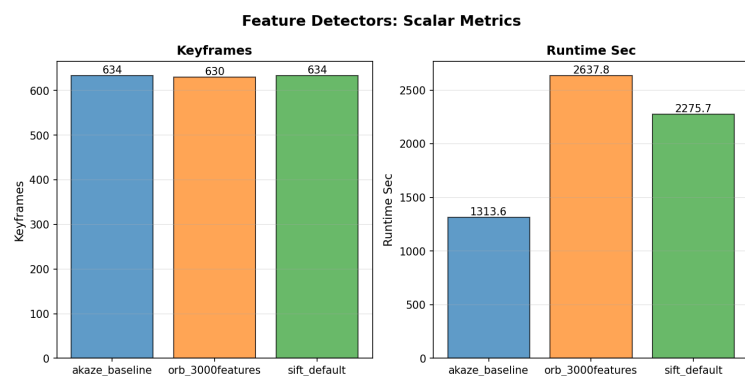


5. Feature Detectors

Compares AKAZE (baseline), ORB (3,000 features), and SIFT.



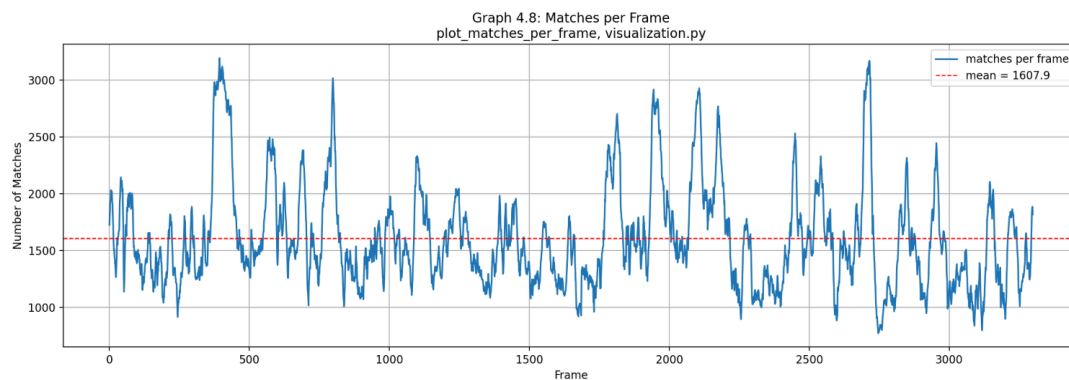
AKAZE demonstrates superior overall performance, delivering the optimal balance between computational efficiency and localization accuracy. It achieved the fastest execution time (1313.6 seconds, roughly 2 times faster than ORB and 1.7 times faster than SIFT) while consistently maintaining low absolute location error. Conversely, extracting 3,000 features with ORB significantly increased matching and optimization overhead (2637.8 seconds) without yielding accuracy gains. Thus, AKAZE demonstrates the best balance between computational efficiency and localization accuracy.



3.2 Tracking Statistics

Metric	Value (Function, File, Line)
Total Tracks	901,081 TrackingDB.track_num(), code/tracking_database.py, line 68
Total Frames	3,300 TrackingDB.frame_num(), code/tracking_database.py, line 72
Mean Track Length	4.40 frames compute_tracking_statistics(), code/project/utils/tracking.py, line 75
Mean Frame Links	1,200.94 tracks per frame compute_tracking_statistics(), code/project/utils/tracking.py, line 75
Matches Per Frame	Graph: task_4_8_matches_per_frame.png plot_matches_per_frame(), code/project/utils/visualization.py, line 787
Inlier Percentage	Graph: task_4_5_inlier_percentage.png plot_inlier_percentage(), code/project/utils/visualization.py, line 256

Matches Per Frame:

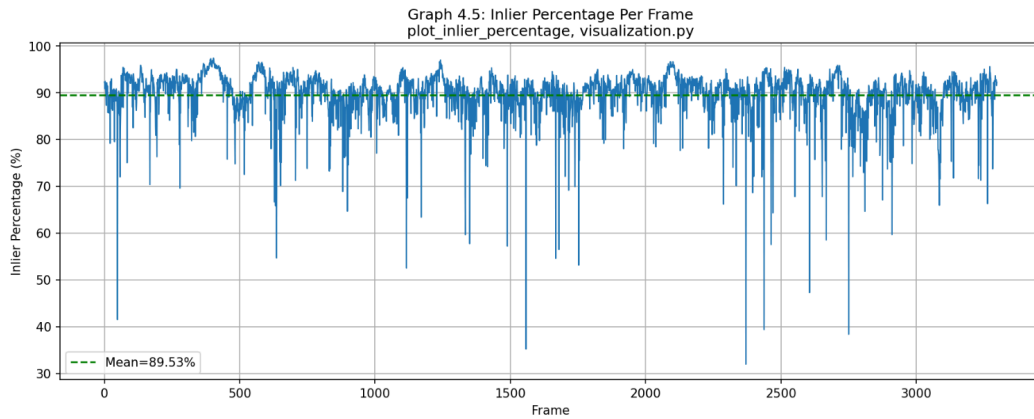


GRAPH ANALYSIS:

This plot displays the total number of feature matches per frame across the sequence:

- **Positive Aspect:** The front-end maintains a strong average feature density of 1607.9 matches per frame (peaking over 3000 matches), providing a rich constraint set for reliable frame-to-frame estimation.
- **Negative Aspect:** Match counts fluctuate noticeably throughout the trajectory, dropping below 800 matches during feature-sparse scenes or fast motion intervals.

Inlier Percentage:



GRAPH ANALYSIS:

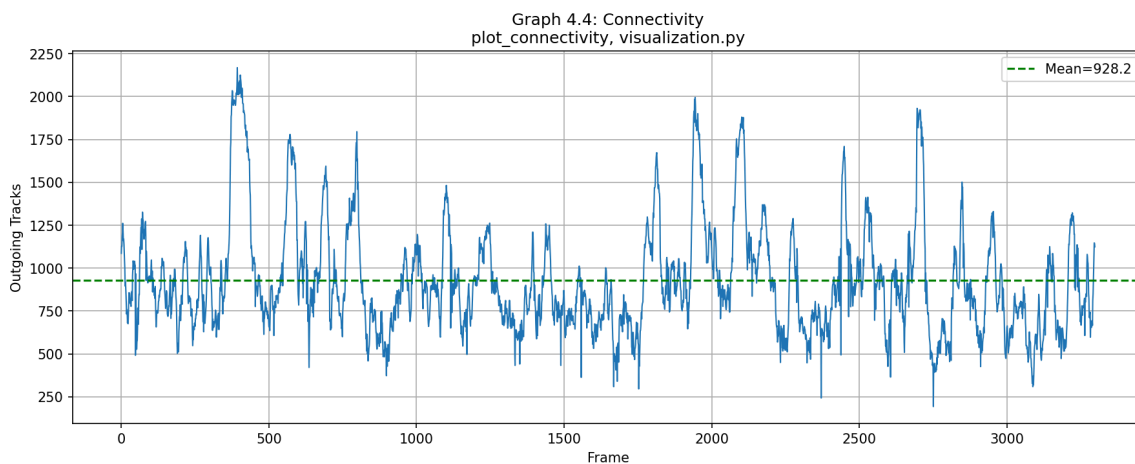
This plot displays the percentage of RANSAC inlier feature matches per frame across the sequence:

- **Positive Aspect:** The system achieves a high mean inlier ratio of 89.53%, demonstrating highly reliable feature tracking and robust pose estimation over time.
- **Negative Aspect:** Occasional sharp dips (dropping down to 30–40% at specific frames) reveal localized tracking sensitivity during rapid vehicle turns or sudden motion changes.

3.3 Key Performance Graphs

Figure 1: Connectivity

Frame-to-frame track linkage counts | Function: plot_connectivity() | File: code/project/utis/visualization.py | Line: 243



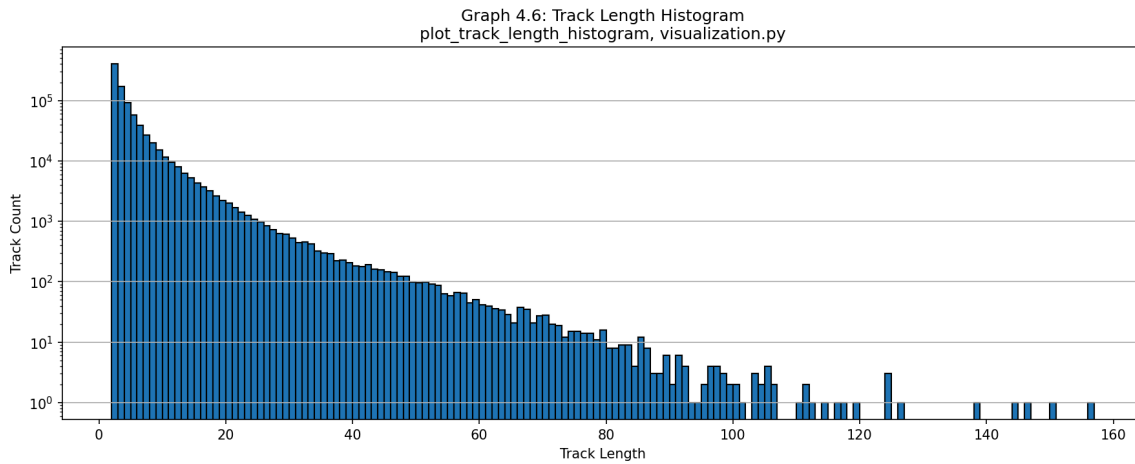
GRAPH ANALYSIS:

The connectivity graph shows the number of active feature tracks continuing from each frame to the subsequent frame throughout the dataset. On average, the system maintains a high continuity of **928.2 outgoing tracks per frame**, demonstrating strong temporal persistence and database linkage across consecutive frames.

- **Positive Aspect:** The baseline connectivity remains well above 250 tracks even during troughs, ensuring that PnP estimation and the tracking database always have sufficient correspondences to maintain localized pose constraints without frame-to-frame tracking failures.
- **Negative Aspect:** Sharp dips occur during rotational maneuvers, high angular velocities, or transient visual occlusions, which accelerate feature displacement and cause rapid track termination across consecutive frames.

Figure 2: Track Length Histogram

Distribution of feature track lifespans (log scale) | Function: `plot_track_length_histogram()` |
File: `code/project/utils/visualization.py` | Line: 270



GRAPH ANALYSIS:

This logarithmic histogram shows the lifespan distribution of feature tracks, characterized by a steep exponential decay where the majority of tracks persist for short durations (2–10 frames).

- **Positive Aspect:** The presence of a long tail reaching up to 160 frames demonstrates the system's ability to maintain long-term feature linkages, which provides strong co-visibility constraints for Bundle Adjustment.
- **Negative Aspect:** The massive concentration of very short tracks highlights frequent feature turnover caused by dynamic motion, occlusions, or image boundary exits.

Figure 3: Trajectory Comparison (Bird's Eye)

Overlay of PnP, Bundle, Pose-Graph+LC, and ground-truth trajectories | Function: `plot_full_trajectory_comparison()` | File: `code/project/utils/visualization.py` | Line: 893

GRAPH ANALYSIS: This figure compares the estimated vehicle trajectories against the Ground Truth (green line) in a bird's-eye view (X-Z plane).

- **Positive Aspect:** Local Bundle Adjustment (blue dots) reduces drift locally, while Pose-Graph Optimization with Loop Closure (orange triangles) successfully eliminates global accumulated error upon revisiting locations, closely aligning the trajectory with the Ground Truth.
- **Negative Aspect:** The pure PnP trajectory (blue line) accumulates severe drift over time, diverging significantly from the true path due to unconstrained frame-to-frame error accumulation.

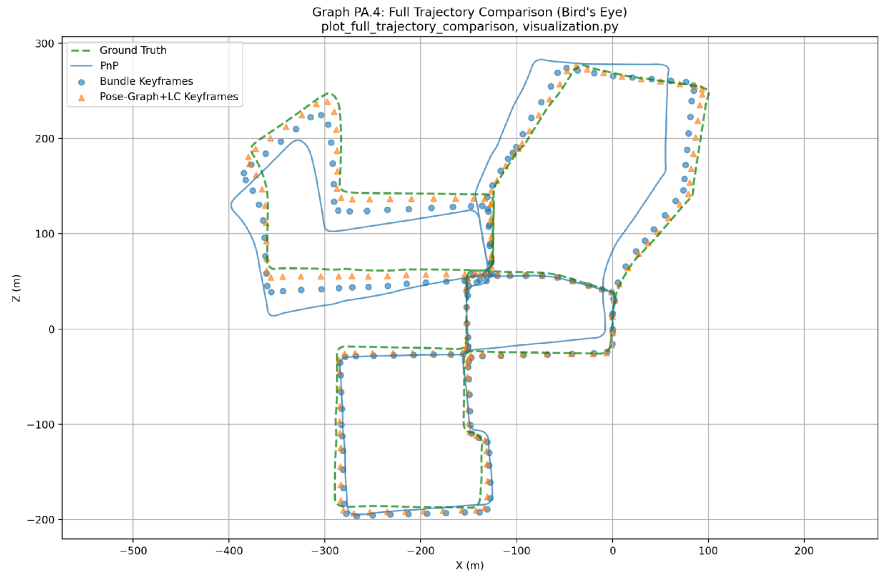
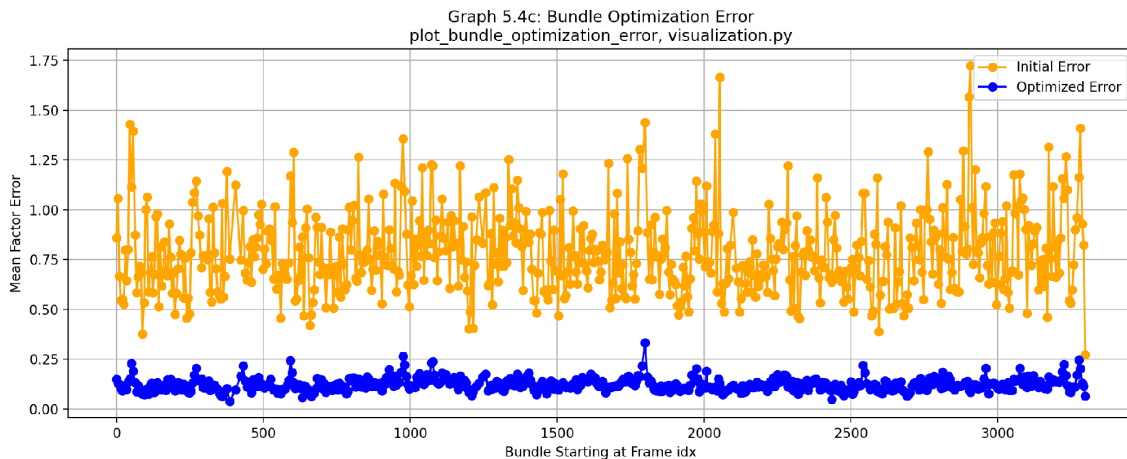


Figure 4: Bundle Optimization Error

Mean factor-graph error before/after optimization per keyframe window | Function: `plot_bundle_optimization_error()` | File: `code/project/utils/visualization.py` | Line: 835



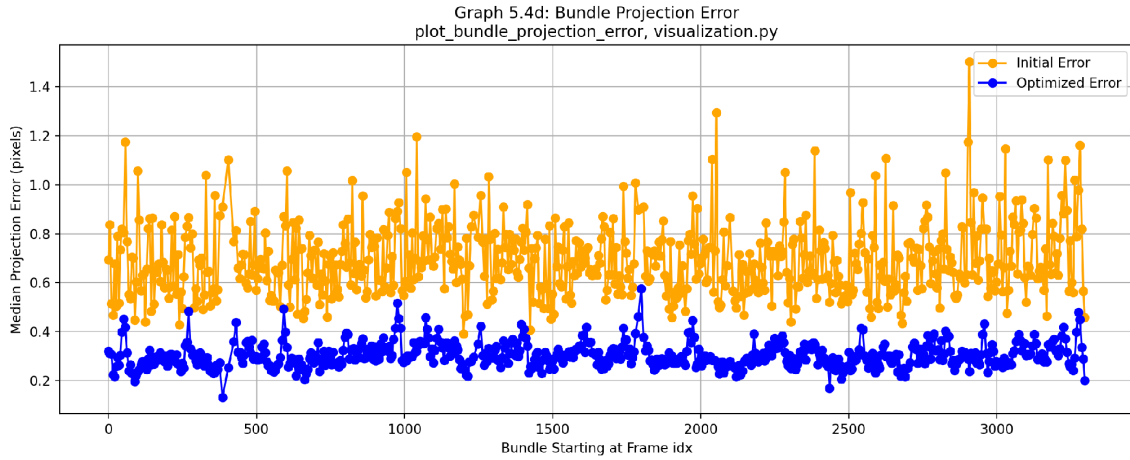
GRAPH ANALYSIS: This plot compares the mean factor-graph error before (orange) and after (blue) optimization for every bundle window. The most notable observation is that, despite considerable variation in the initial error, the optimized error remains consistently low across almost all windows.

- **Positive Aspect:** Bundle Adjustment consistently reduces the optimization error from approximately 0.5–1.75 to below 0.25 for nearly every bundle window, demonstrating stable convergence and effective refinement of camera poses and landmarks.

- **Negative Aspect:** Several windows exhibit substantially higher initial errors than the rest of the sequence. Although optimization successfully corrects these windows, the variation indicates that some local windows begin from less accurate initial estimates than others.

Figure 5: Bundle Projection Error

Median projection error of landmarks before/after optimization | Function: `plot_bundle_projection_error()` | File: `code/project/utils/visualization.py` | Line: 850



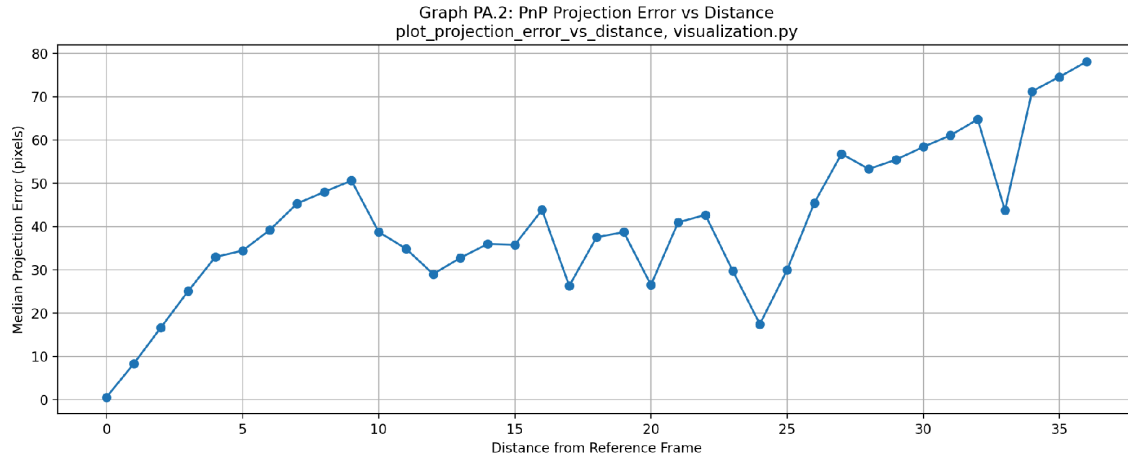
GRAPH ANALYSIS: This graph presents the median reprojection error before and after Bundle Adjustment for each optimization window.

- **Positive Aspect:** Bundle Adjustment consistently reduces the median reprojection error to approximately 0.2–0.4 pixels, achieving sub-pixel accuracy throughout the sequence.
- **Negative Aspect:** The initial reprojection error varies noticeably between optimization windows, indicating that the quality of the initial geometric estimates is not uniform across the trajectory. Nevertheless, optimization converges to a consistently low residual in nearly every case.

Figure 6: Track Link Error vs. Distance (PnP)

Projection error as function of track-point distance (PnP method) | Function: `plot_projection_error_vs_distance()` | File: `code/project/utils/visualization.py` | Line: 865

Track Sampling Method. To reduce computational cost while preserving statistical representativeness, the projection-error analysis was performed on a uniform random sample of up to 150 tracks with a minimum length of 5 frames. A fixed random seed (`seed = 0`) was used to ensure reproducibility. This subset was selected from the complete tracking database (901,081 tracks) and provides a representative estimate of the relationship between projection error and track distance.



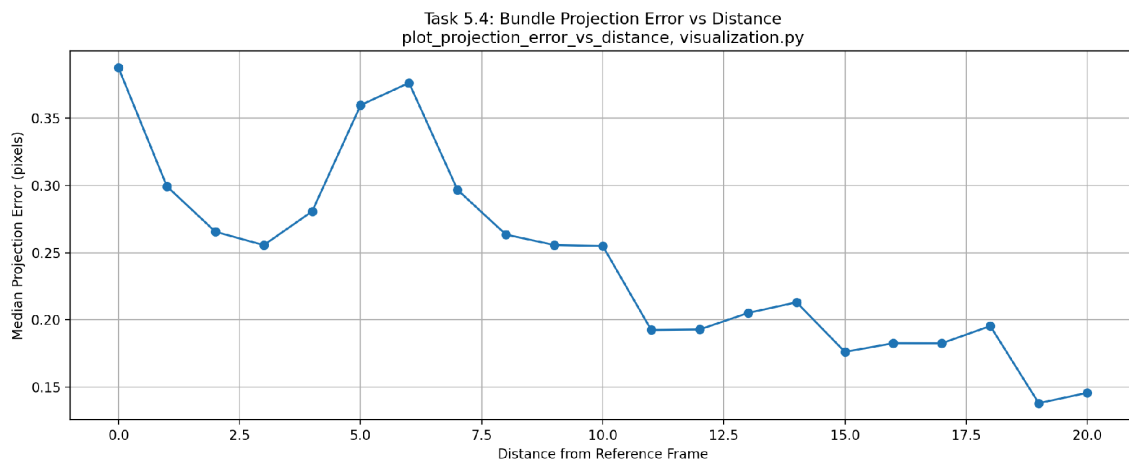
GRAPH ANALYSIS: This graph shows the median reprojection error of 3D feature points in PnP estimation as a function of frame distance from their initial triangulation reference frame.

- **Negative Aspect:** The error exhibits a clear upward drift trend, climbing from sub-pixel accuracy near zero distance to around 80 pixels at distance 36 likely reflecting accumulated geometric distortion and pose drift.
- **Positive Aspect:** The error remains low and manageable (below 35 pixels) for short track links (0–10 frames), confirming that PnP frame-to-frame estimation is reliable over short sliding windows before Bundle Adjustment is required.

Figure 7: Track Link Error vs. Distance (Bundle)

Projection error as function of track-point distance (Bundle method) | Function: `plot_projection_error_vs_distance()` | File: `code/project/utils/visualization.py` | Line: 865

Sampling Method. Unlike the PnP analysis, Bundle Adjustment evaluates **all optimized landmarks** (`optimized_landmark_ids`) within every local bundle-adjustment window. Since each bundle contains only a limited number of landmarks, no subsampling is required, eliminating sampling bias.



GRAPH ANALYSIS: This plot displays the median reprojection error of 3D points optimized via Bundle Adjustment as a function of frame distance from their reference frame:

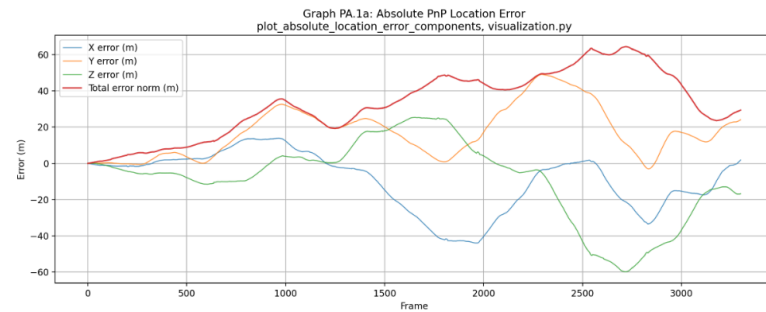
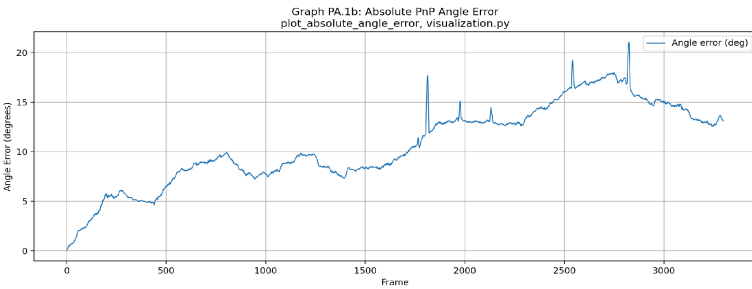
- **Positive Aspect:** Bundle Adjustment successfully suppresses reprojection error growth across frame distances, maintaining sub-pixel accuracy below 0.4 pixels throughout and stabilizing below 0.25 pixels for longer track distances.
- **Negative Aspect:** A slight initial variation occurs at short frame distances (0–6 frames) due to initial triangulation adjustments before the optimization stabilizes across the sequence.

Figure 8: Absolute PnP Error

- **Figure 8A: Absolute Position Error (X, Y, Z, Norm)**
- **Figure 8B: Absolute Angle Error**

X/Y/Z position error and angle error vs. ground truth (PnP) | Function: `plot_absolute_angle_error()` | File: `code/project/utis/visualization.py` | Line: 821

X/Y/Z position error and total location error norm (meters). Function: `plot_absolute_location_error_components()` | File: `code/project/utis/visualization.py` | Line: 804



GRAPH ANALYSIS: These plots present the absolute position error (X, Y, Z and total norm) together with the absolute orientation error of the raw PnP trajectory.

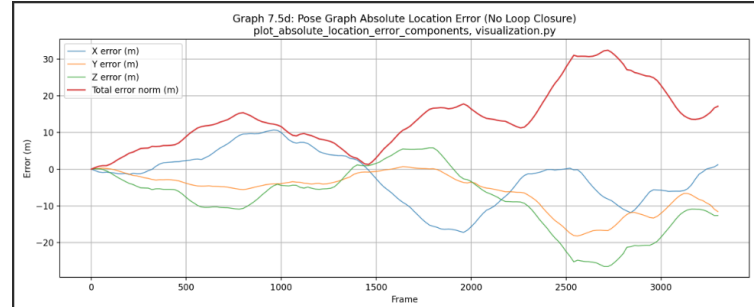
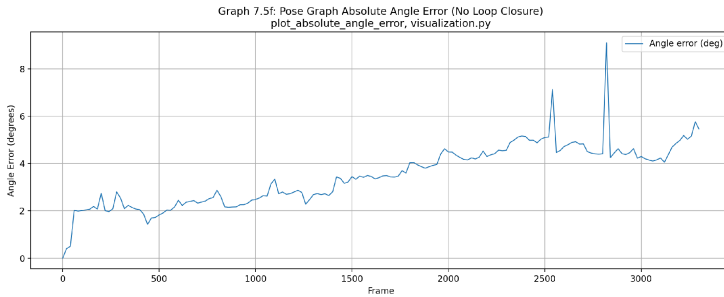
- **Positive Aspect:** During the early part of the sequence, both position and orientation errors remain relatively small, indicating that frame-to-frame PnP estimation provides accurate short-term motion estimates.
- **Negative Aspect:** The total position error increases steadily throughout the trajectory, reaching approximately 65 m by the end of the sequence. The orientation error exhibits a similar cumulative trend with several sharp peaks. Since PnP estimates only the motion between consecutive frames without any global optimization, small local errors accumulate over time, resulting in significant drift.

Figure 9: Absolute Pose-Graph Error (No Loop Closure)

- **Figure 9A: Absolute Position Error (X, Y, Z, Norm)**
- **Figure 9B: Absolute Angle Error**

Absolute error before loop-closure integration | Function: `plot_absolute_angle_error()` | File: `code/project/utis/visualization.py` | Line: 821

X/Y/Z position error and total location error norm (meters), post-Bundle Adjustment. Function: `plot_absolute_location_error_components()` | File: `code/project/utils/visualization.py` | Line: 804



GRAPH ANALYSIS: These plots evaluate the absolute position and orientation errors after Bundle Adjustment and Pose-Graph optimization without loop closure.

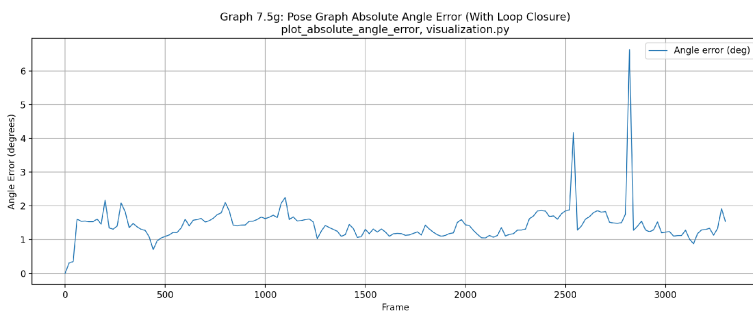
- **Positive Aspect:** Compared with the raw PnP trajectory, the pose graph significantly reduces both translational and rotational drift. Position errors remain substantially lower throughout the sequence, demonstrating the effectiveness of local bundle constraints.
- **Negative Aspect:** Although the growth rate of the error is considerably reduced, the error still increases gradually over long distances because no global constraints exist to correct accumulated drift.

Figure 10: Absolute Pose-Graph Error (With Loop Closure)

- **Figure 10A: Absolute Position Error (X, Y, Z, Norm)**
- **Figure 10B: Absolute Angle Error**

Absolute error after loop-closure optimization shows improvement | Function: `plot_absolute_angle_error()` | File: `code/project/utils/visualization.py` | Line: 821

X/Y/Z position error and total location error norm (meters), post-bundle adjustment and loop-closure optimization. Function: `plot_absolute_location_error_components()` | File: `code/project/utils/visualization.py` | Line: 804



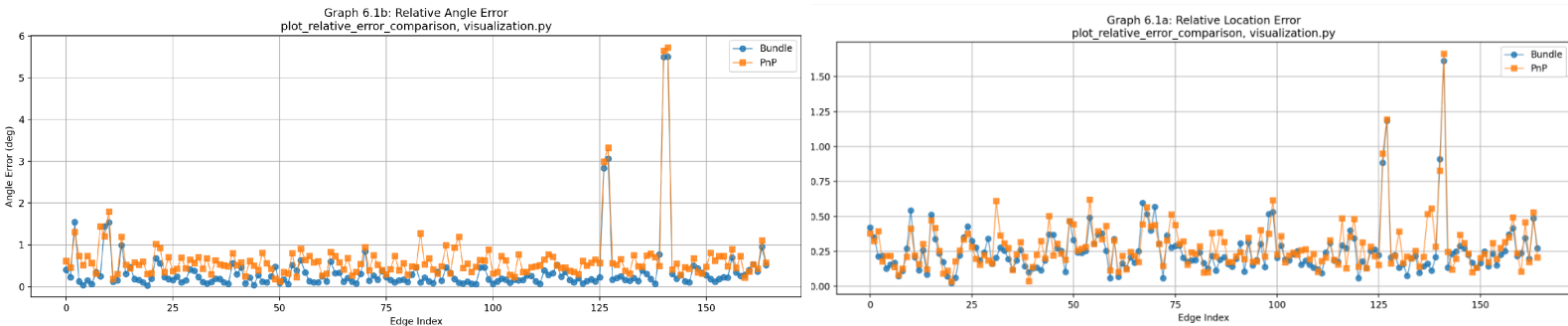
GRAPH ANALYSIS: Incorporating loop-closure constraints produces the lowest overall localization error among all evaluated methods.

- **Positive Aspect:** Loop closures effectively redistribute accumulated drift throughout the pose graph, reducing the maximum position error from approximately 32 m to about 11 m while also maintaining a consistently low orientation error.
- **Negative Aspect:** Small localized peaks remain between loop-closure events, indicating that correction is strongest near revisited locations and weaker in regions without additional global constraints.

Figure 11: Relative Error Between Consecutive Keyframes

- **Figure 11A: Relative Location Error (Bundle vs. PnP)**
- **Figure 11B: Relative Angle Error (Bundle vs. PnP)**

Per-edge relative location and pose error (m) comparing Bundle Adjustment and PnP to ground truth between consecutive keyframe windows, visualized via `plot_relative_error_comparison()` (code/project/utls/visualization.py, Line



878).

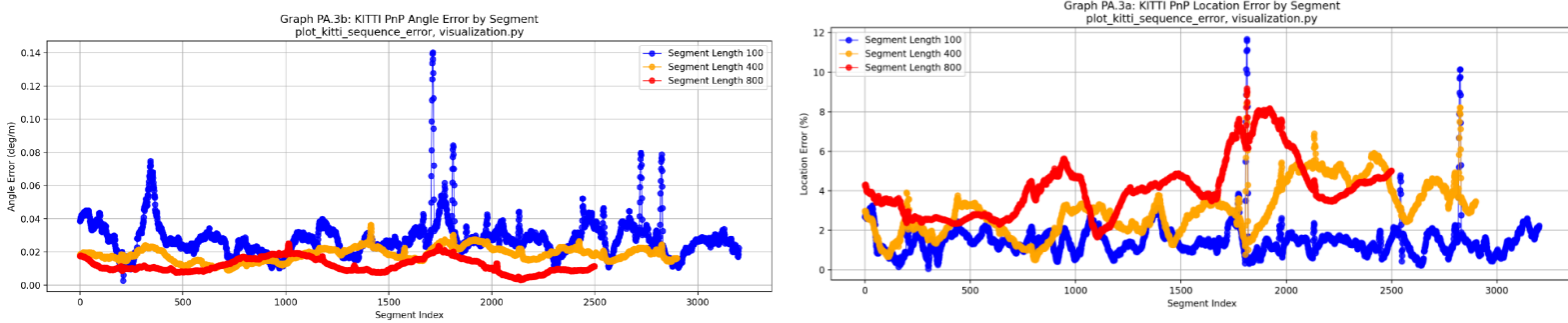
GRAPH ANALYSIS: These plots compare relative angle error (degrees, left) and relative location error (meters, right) between consecutive keyframes for Bundle Adjustment (blue circles) and PnP (orange squares):

- **Positive Aspect:** Bundle Adjustment consistently outperforms PnP across almost all keyframe edges, maintaining a tight baseline well below 0.5 degrees for rotational error and achieving a low mean location error of 0.2563 m through multi-frame factor graph optimization.
- **Negative Aspect:** Both methods exhibit identical co-occurring error spikes at edge indices ~127 (3.0° / 1.0–1.2 m) and ~140 ($\sim 5.7^\circ$ / 1.6–1.85 m). Because PnP (frame-by-frame) and Bundle Adjustment (windowed optimization) rely on fundamentally different estimation pipelines, their near-identical divergence suggests that localized inaccuracies or noise in the ground-truth annotations may contribute to the observed error spikes, rather than indicating a failure of the Visual-SLAM pipeline.

Figure 12: Relative PnP Error (KITTI Sub-segments)

- Figure 12A: Relative Translation Error for Sub-segments of Length 100, 400, and 800
- Figure 12B: Relative Angle Error for Sub-segments of Length 100, 400, and 800

Translation error (%) for relative PnP poses evaluated across fixed-length sliding windows of 100, 400, and 800 frames, visualized via `plot_kitti_sequence_error()` (code/project/utls/visualization.py, Line 915).



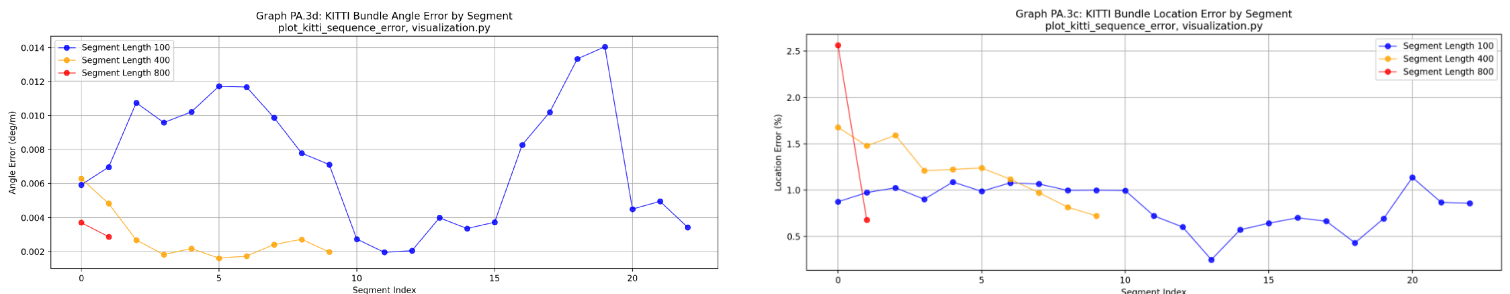
GRAPH ANALYSIS: These plots evaluate relative PnP angle error (deg/m, left) and location error (% , right) across KITTI sub-segments of lengths 100, 400, and 800 frames:

- **Positive Aspect:** Evaluation across window lengths captures key resolution dynamics: short 100-frame windows maintain a strong translation baseline (averaging 1.48%), while longer windows smooth out short-term angular fluctuations, yielding lower average rotational error rates (decreasing from 0.0280 deg/m at 100 frames to 0.0120 deg/m at 800 frames).
- **Negative Aspect:** Cumulative unconstrained drift scales translation error over longer distances (averaging 3.05% for 400 frames and 4.13% for 800 frames), while short-term tracking sensitivity produces severe co-occurring spikes reaching ~0.14 deg/m in angle error and ~12% in location error around segment indices ~1800 and ~2800.

Figure 13: Relative Bundle Error (KITTI Sub-segments)

- Figure 13A: Relative Translation Error for Sub-segments of Length 100, 400, and 800
- Figure 13B: Relative Angle Error for Sub-segments of Length 100, 400, and 800

Translation error (%) for relative Bundle-optimized keyframe poses evaluated across fixed-length sliding windows of 100, 400, and 800 frames, visualized via `plot_kitti_sequence_error()` (code/project/utls/visualization.py, Line 915).



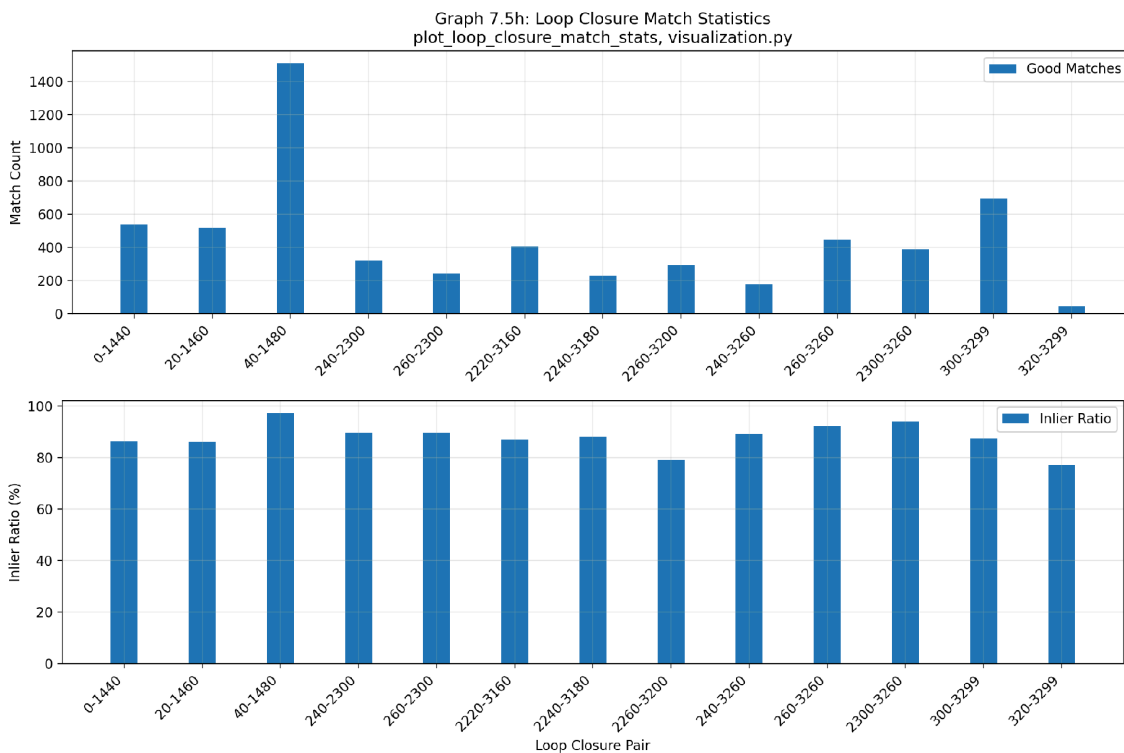
GRAPH ANALYSIS: These plots evaluate relative Bundle Adjustment angle error (deg/m, left) and location error (% , right) across KITTI sub-segments of lengths 100, 400, and 800 keyframe windows:

- **Positive Aspect:** Bundle Adjustment achieves superior trajectory precision, maintaining remarkably low average rotational errors (0.0073 deg/m for 100-window, 0.0028 deg/m for 400-window) alongside a low and flat location error profile (averaging 0.83% for 100-window and 1.20% for 400-window).
- **Negative Aspect:** Keyframe sparsity (166 keyframes vs. 3,300+ total frames) results in very few sample points for long 800-window evaluations (reducing statistical confidence), while short 100-window segments exhibit localized angular fluctuations up to ~0.014 deg/m likely caused by short-term geometric inconsistencies.

Figure 14: Loop-Closure Match Statistics

Number of inliers per successfully verified loop closure | Function:

plot_loop_closure_match_stats() | File: code/project/utis/visualization.py | Line: 945



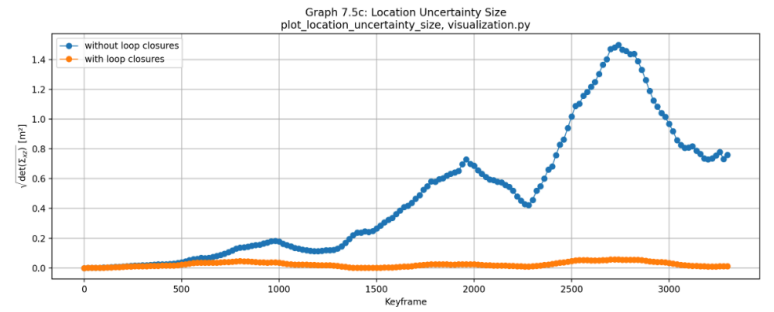
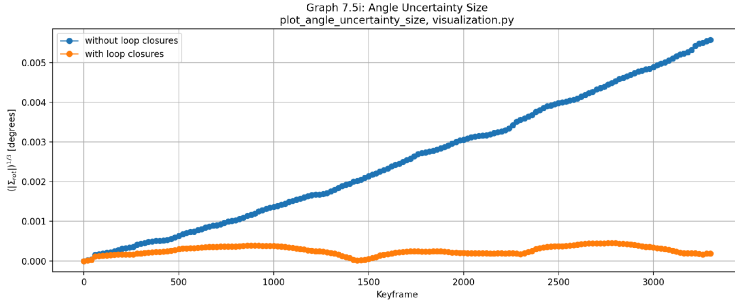
GRAPH ANALYSIS: This figure shows the number of good matches and inlier ratios across detected loop closure pairs:

- **Positive Aspect:** The system maintains exceptional matching quality, with inlier ratios consistently exceeding 80% across all verified pairs (peaking at ~97% for pair 40-1480), confirming robust outlier rejection during geometric verification.
- **Negative Aspect:** Total match counts fluctuate significantly across loop pairs, ranging from over 1500 matches down to under 50 for pair 320-3299, reflecting varying visual overlap and perspective changes when revisiting locations.

Figure 15: Uncertainty Size (Location & Angle)

Covariance-derived uncertainty estimates with and without loop closure | Function: `plot_angle_uncertainty_size()` | File: `code/project/utils/visualization.py` | Line: 975

Location uncertainty (m^2) as a function of keyframe index, for pose-graph optimization with and without loop-closure constraints. Function: `plot_location_uncertainty_size()` | File: `code/project/utils/visualization.py` | Line: 749



Location uncertainty is defined as:

$$Uncertainty_{loc} = \sqrt{\det(\Sigma_{xz})}$$

where $\Sigma_{xz} \in R^{2 \times 2}$ is the XZ sub-block of the full 6×6 pose covariance matrix, extracted via `marginals.marginalCovariance()` (`visualization.py:764-765`).

Rotational (angle) uncertainty is defined as:

$$Uncertainty_{rot} = \det(\Sigma_{rot})^{\frac{1}{3}} \times \frac{180}{\pi} \quad [degrees]$$

where $\Sigma_{rot} \in R^{3 \times 3}$ is the rotational sub-block of the covariance matrix (`visualization.py:984-987`).

The uncertainty of each optimized pose is obtained from the marginal covariance matrix computed by GTSAM after optimization. The resulting 6×6 covariance matrix contains both translational and rotational uncertainty.

To analyze these quantities independently, the covariance matrix is partitioned into its translational (upper-left 3×3 block) and rotational (lower-right 3×3 block) components.

For each component, uncertainty size is measured using $\sqrt{\det(\Sigma)}$ where Σ is the corresponding covariance submatrix.

The determinant represents the volume of the uncertainty ellipsoid, while taking the square root restores the physical units of the corresponding variables. This provides a single scalar quantity that summarizes the overall uncertainty while accounting for correlations between all three dimensions.

GRAPH ANALYSIS: These plots compare covariance-derived rotational and location uncertainties across keyframes, evaluated with (orange) and without (blue) loop closures:

- **Positive Aspect:** Integrating loop closure constraints dramatically suppresses overall pose uncertainty, keeping rotational uncertainty strictly bounded below 0.0005 degrees and achieving a ~20 times reduction in spatial uncertainty (under $0.08 m^2$) by redistributing confidence globally.

- **Negative Aspect:** Without loop closures, uncertainty accumulates monotonically over time as odometry drift compounds, peaking above 0.0055 degrees for orientation and reaching 1.5 m^2 for location around keyframes 2700–2800.

4. Discussion and Conclusions

4.1 Summary and Critique

Overall, the experimental evaluation demonstrates a consistent improvement in localization accuracy throughout the pipeline. Frame-to-frame PnP provides reliable short-term pose estimation but accumulates significant drift over long trajectories. Bundle Adjustment substantially improves local geometric consistency, Pose-Graph optimization further reduces accumulated error, and Loop Closure provides the largest improvement by correcting global drift. The close agreement between the quantitative error metrics and the trajectory comparisons validates the correctness and robustness of the implemented Visual-SLAM system.

Key Strengths

1. **Robust Temporal Tracking:** Consistent observation linkage over time is maintained through a dedicated tracking database, providing reliable feature correspondences across consecutive frames.
2. **Multi-Stage Outlier Rejection:** Effective noise filtering is achieved through multiple complementary stages, including PnP-RANSAC, sliding-window Bundle Adjustment, and loop-closure verification.
3. **Principled Uncertainty Quantification:** Estimation confidence is rigorously evaluated using covariance matrices derived from GTSAM, enabling quantitative analysis of both translational and rotational uncertainty throughout the optimization process.

Limitations

1. **Scene Revisitation Dependency:** Loop closure detection is limited to previously visited locations and therefore requires sufficient physical trajectory overlap. As a result, environments with few revisited areas offer fewer opportunities for global drift correction.
2. **Fixed Optimization Window:** The Bundle Adjustment window size remains fixed throughout the trajectory. While computationally efficient, a static window may not be optimal for environments with varying feature density or scene complexity.
3. **Dynamic Scene Assumption:** The current pipeline assumes a predominantly static environment and does not explicitly detect or reject moving objects. In highly dynamic scenes, transient features from moving objects may reduce tracking stability and localization accuracy.

4.2 Future Work

To further enhance the system's robustness against potential outlier loop closures, especially in complex or highly dynamic environments, future work can explore adaptive and robust back-end optimization techniques from the literature. Rather than relying solely on strict, hard-threshold pre-optimization gating, these approaches enable continuous, residual-dependent down-weighting during non-linear factor graph optimization.

Switchable Constraints (Sünderhauf & Protzel, IROS 2012)

Future extensions could integrate **Switchable Constraints** to dynamically evaluate loop closure validity during optimization. This method introduces a continuous scalar switch variables $s_i \in [1, 0]$ for each loop closure edge, which is optimized jointly alongside the robot poses. By modulating constraint weights, where values near 1.0 endorse a valid loop closure and values approaching 0.0 deactivate potential outliers, the system can gracefully suppress false-positive constraints without discarding useful geometric priors prematurely.

Dynamic Covariance Scaling (Agarwal et al., ICRA 2013)

Another promising direction is **Dynamic Covariance Scaling (DCS)**, which performs residual-dependent reweighting of the information matrix. Under this framework, constraints exhibiting large chi-squared (χ^2) residuals are automatically down-weighted during optimization. By applying an adaptive scaling function (e.g., with a chi-squared threshold of $\Phi = 1.0$), DCS provides a closed-form, computationally efficient alternative to robust loss functions, preventing erroneous loop closures from distorting the trajectory estimate.

Appendix: Relative Error Formulas

This appendix presents the mathematical formulations for angle and segment-based relative error, which are fundamental to evaluating the accuracy of pose estimation throughout the pipeline.

Rodrigues Representation for Rotation Error

Comparing the rotation difference between estimated and ground-truth relative poses is best done with the Rodrigues formula, which represents a rotation as a 3D vector. The direction of the vector is the axis of rotation, and the magnitude is the rotation angle in radians.

For a rotation matrix R , the angle magnitude in degrees is computed as:

$$\theta_{deg} = \|\mathbf{r}\|_2 \times \frac{180}{\pi} \quad (1)$$

This is implemented in `code/project/utils/geometry.py:87–90` (function `rotation_angle_deg`).

Relative Pose Error over KITTI Segments

The KITTI benchmark evaluates accuracy on fixed-length trajectory segments. For a segment from frame i to frame j , relative pose error is normalized by the total ground-truth path length traveled, not the direct distance.

- **Location Error (% of path length):**

$$Error_{loc} = \frac{\|\Delta \mathbf{p}_{i,j}\|_2}{L_{gt}(i,j)} \times 100\% \quad (2)$$

- **Angle Error (degrees per meter):**

$$Error_{rot} = \frac{|\Delta \theta_{i,j}|}{L_{gt}(i,j)} \quad (3)$$

Where $L_{gt}(i, j)$ represents the cumulative ground-truth trajectory path length:

$$L_{gt}(i, j) = \sum_{k=i}^{j-1} \|\mathbf{c}_{k+1} - \mathbf{c}_k\|_2 \quad (4)$$

This evaluation pipeline is implemented in `code/project/utils/geometry.py:109–131` (`compute_kitti_sequence_errors`).